

BMI Calculator CI/CD Pipeline

Comprehensive Architecture, Technology Rationale & Viva Master Guide

1. Executive Summary & Problem Statement

Problem Statement: Create a local **Jenkins** pipeline that triggers a **Docker** build, runs **Ansible** scripts to check container vulnerabilities, and deploys the app to **K3s (Kubernetes)** using **Terraform** scripts.

This project models an enterprise-grade, shift-left DevOps delivery pipeline. It unites application development (Python/Flask), containerization (Docker), continuous integration (Jenkins), automated security governance (Ansible + Trivy), declarative Infrastructure-as-Code (Terraform), and container orchestration (K3s Kubernetes).

Pipeline Stage	Technology	Core Responsibility
1. Application	Python 3.11 / Flask	Calculates BMI and provides /health probe endpoint.
2. Packaging	Docker (multi-layer)	Packages app into immutable image with non-root security.
3. Orchestration	Jenkins (Declarative)	Executes stages sequentially and enforces fail-fast error gating.
4. Security Scan	Ansible + Trivy	Scans image for CVEs; aborts pipeline if CRITICAL/HIGH found.
5. Provisioning	Terraform (IaC)	Pushes desired state (2 replicas, Service) to K3s cluster.
6. Runtime	K3s (Kubernetes)	Executes pods, manages self-healing, routes traffic via NodePort.

2. Deep Dive: Why This Tool and NOT That?

A. Python Flask (Application Layer)

- **What it is:** A lightweight WSGI microframework for Python designed for rapid web applications and microservices.
- **Why used here:** Contains minimal overhead, boots instantly (<0.5s), and exposes both the BMI calculation UI and a '/health' JSON probe.
- **Why NOT Django?** Django is a heavyweight, full-stack monolith containing ORM, admin panel, and session engines unnecessary for a microservice. It bloats container image size.
- **Why NOT Express/Node.js?** Python is the ubiquitous language of cloud computing and Linux administration; Flask delivers the smallest attack surface.

B. Docker (Containerization Layer)

- **What it is:** An operating-system-level virtualization platform that packages software into immutable containers.
- **Why used here:** Guarantees reproducible runtime environments, packaging Python, Flask, and dependencies together. Solves 'it works on my machine.'
- **Why NOT Virtual Machines (VMs)?** A VM virtualizes the entire hardware stack and boots a guest OS (GBs of memory, minutes to boot). Docker containers share the host Linux kernel (MBs of memory, seconds to boot).

- **Security Highlight:** The Dockerfile employs layer caching (pip install requirements before copying code) and enforces 'USER appuser' so processes never execute as root.

C. Jenkins (Continuous Integration & Orchestration)

- **What it is:** An open-source automation server enabling automated building, testing, and deploying via Pipeline-as-Code (Jenkinsfile).
- **Why used here:** Acts as the pipeline master coordinator. Executes each stage sequentially, prints live console logs, and aborts before deployment if tests fail.
- **Why NOT GitHub Actions / GitLab CI?** GitHub Actions runs on cloud runners, requiring internet connectivity, billing credits, and complex tunneling to access local K3s clusters. Jenkins runs 100% locally on-premises without cloud dependencies.
- **Why NOT raw bash scripts?** Bash scripts lack visual dashboards, stage-by-stage retry capabilities, built-in credential management, and structured failure reporting.

D. Ansible + Trivy (Shift-Left Security Gate)

- **What it is:** Ansible is an agentless automation engine. Trivy is a comprehensive vulnerability scanner for container images.
- **Why used here:** Ansible executes a playbook that downloads Trivy if missing, scans the built image for CRITICAL/HIGH vulnerabilities, and fails with exit code 1 if CVEs are found.
- **Why NOT a direct shell command?** Ansible playbooks are idempotent, written in human-readable YAML, support structured output registration, and can effortlessly scale to scan across multiple remote registry nodes.
- **Why NOT Clair or Snyk?** Snyk requires proprietary cloud account keys and rate-limits free tiers. Clair requires an external PostgreSQL database. Trivy is a zero-dependency standalone binary.

E. Terraform (Infrastructure as Code - IaC)

- **What it is:** A declarative Infrastructure-as-Code tool that provisions and manages resources across cloud providers and Kubernetes clusters.
- **Why used here:** Terraform connects directly to the K3s cluster API via the HashiCorp Kubernetes provider, creating the Deployment (2 replicas) and NodePort Service.
- **Why NOT 'kubectl apply -f'?** Kubectl is imperative and stateless—it pushes files without tracking state, cannot preview changes ('plan'), and cannot detect configuration drift. Terraform tracks reality in 'terraform.tfstate' and manages the full resource lifecycle.
- **Why NOT Helm?** Helm is a package manager for pre-built charts. Terraform manages foundational infrastructure and seamlessly passes dynamic variables (such as new image tags) into Kubernetes definitions.

F. K3s (Container Orchestration Layer)

- **What it is:** A lightweight, CNCF-certified Kubernetes distribution packaged as a single binary under 100MB, created by Rancher.
- **Why used here:** Runs the production containers, handles self-healing (restarting crashed pods), load balances traffic between the 2 replicas, and monitors pod health.
- **Why NOT full Kubernetes (Kubeadm)?** Kubeadm requires at least 4GB of RAM, multiple system services, and complex etcd management. K3s replaces etcd with SQLite/embedded storage, consuming less than 512MB.

RAM while remaining 100% compliant with Kubernetes APIs.

- **Why NOT Minikube?** Minikube is strictly a development sandbox. K3s is a true production-grade distribution deployed globally in edge, IoT, and lightweight server environments.

3. Technology Comparison Matrix (Viva Cheat Sheet)

Component	Choice	Alternative	Why Our Choice Won
Framework	Flask	Django	Zero boilerplate, instant boot, minimal memory footprint.
Runtime	Docker	Virtual Machine	Kernel sharing: 10x lighter, boots in seconds, standard packaging.
CI/CD	Jenkins	GitHub Actions	100% local, self-hosted, no cloud tokens or internet needed.
Security	Ansible+Trivy	Shell Script	Idempotent, YAML-based, structured enterprise scanner pattern.
Scanner	Trivy	Snyk / Clair	Single binary, no database required, instant CVE check.
IaC	Terraform	kubectl apply	State tracking (tfstate), diff preview (plan), drift detection.
Orchestrator	K3s	Kubeadm	CNCF-compliant, <100MB binary, 80% lower RAM footprint.

4. Top 15 Viva Questions & Word-for-Word Answers

Practice these concise answers to achieve top marks during your viva or project presentation:

Q1: Explain the high-level architecture of your project in one sentence.

Answer: Jenkins automates the pipeline: it builds a Docker image of our Python BMI web app, uses Ansible to scan that image for vulnerabilities with Trivy, and if it passes, uses Terraform to declare the deployment to a lightweight K3s Kubernetes cluster.

Q2: What is 'Shift-Left Security' and how is it implemented in this project?

Answer: 'Shift-Left Security' means performing vulnerability checks early in the development lifecycle rather than in production. We implement this in Stage 3: Ansible triggers a Trivy scan on the newly built container. If CRITICAL or HIGH vulnerabilities are detected, Trivy exits with code 1, which immediately fails the Jenkins build and blocks deployment.

Q3: What is the difference between a Docker Image and a Container?

Answer: A Docker image is a static, read-only, immutable template containing application code, runtime, and libraries. A container is a running, isolated execution instance created from that image.

Q4: Why did your Dockerfile copy 'requirements.txt' before copying the rest of the application code?

Answer: To take advantage of Docker layer caching. Dependencies in 'requirements.txt' rarely change. By copying and installing them first, Docker reuses the cached layer on subsequent builds, reducing rebuild time from minutes to seconds.

Q5: Why did you add 'USER appuser' inside the Dockerfile?

Answer: Security defense-in-depth. By default, containers run as root. If an attacker breaches the application, running as an unprivileged user prevents them from acquiring root privileges on the underlying host kernel.

Q6: What does 'Infrastructure as Code' (IaC) mean, and why use Terraform over 'kubectl apply'?

Answer: IaC means managing infrastructure via version-controlled configuration files rather than manual commands. Terraform maintains a state file ('terraform.tfstate') to track real-world infrastructure, preview changes via 'terraform plan', detect configuration drift, and ensure declarative idempotency.

Q7: What is the difference between K3s and standard Kubernetes (Kubeadm)?

Answer: K3s is a lightweight, fully compliant, CNCF-certified Kubernetes distribution packaged as a single binary under 100MB. It replaces heavyweight etcd with SQLite and strips out legacy cloud providers, using under 512MB RAM compared to Kubeadm's 4GB+ requirement.

Q8: How does Kubernetes monitor application health in your setup?

Answer: Through Liveness and Readiness probes declared in 'terraform/main.tf'. K3s periodically executes an HTTP GET on '/health' on port 5000. If the application crashes or hangs and fails to return HTTP 200, Kubernetes automatically destroys and restarts the pod.

Q9: What is a Kubernetes Service and why did you select 'NodePort'?

Answer: Pods are ephemeral and get new IP addresses whenever recreated. A Service provides a permanent, stable network IP and load-balances traffic across pods. We selected 'NodePort' (port 30080) because it opens that fixed port on every cluster node, allowing direct web browser access without needing a cloud load balancer.

Q10: What is idempotency and where does it occur in this project?

Answer: Idempotency means that running an operation multiple times produces the exact same end state without duplicate side-effects. Both our Ansible playbook and Terraform scripts are idempotent: if the scan or cluster resources already match desired state, re-running them makes zero changes.

Q11: Why is an Ansible inventory file needed when scanning on localhost?

Answer: Ansible requires an inventory to know target hosts and connection plugins. In 'inventory.ini', setting 'localhost ansible_connection=local' tells Ansible to execute tasks locally on the build machine without trying to establish an SSH connection.

Q12: What happens if Trivy finds a HIGH vulnerability in the container?

Answer: Trivy exits with status code 1 because of the '--exit-code 1' flag. Ansible registers this non-zero exit code as a task failure, triggering the 'fail' module. This causes the Jenkins pipeline step to abort immediately, preventing the push to the registry and deployment to K3s.

Q13: What is the purpose of the local Docker registry running on port 5000?

Answer: In a production environment, images are pushed to Docker Hub or AWS ECR. For our local CI/CD environment, running a local registry ('registry:2') allows Jenkins to push images and K3s to pull images locally without requiring internet access or third-party credentials.

Q14: How does Jenkins pass the dynamic image tag to Terraform?

Answer: Jenkins assigns the current build number to an environment variable ('\${BUILD_NUMBER}'). It passes this to Terraform during the apply stage via the CLI flag: '-var="image_name=localhost:5000/bmi-calculator:\${BUILD_NUMBER}"'. Terraform then updates the Kubernetes deployment definition.

Q15: What are the common viva mistakes to avoid?

Answer: Never confuse containers with virtual machines (containers share the host kernel). Never claim Terraform runs the app (Terraform provisions the K8s objects; K3s runs the containers). Never claim Ansible deploys the app (Ansible executes the security scan; Terraform executes deployment).

End of Master Guide. Best of luck on your presentation and viva!